ITS Notification Library

Integration Guide for Android Apps

ITS Notification Library

An Android SDK that wraps Firebase Cloud Messaging (FCM) push delivery together with ITS's own backend (device registration, notification history, delivery/read-status tracking) behind a small Java API, so a host app doesn't need to talk to either directly.

This guide covers everything needed to integrate the library into an **existing** Android app: setup, required manifest changes, the full API surface, the gotchas that are easy to miss, and the constraints you need to know about before you start.

---

1. Requirements

- `minSdk 23` or higher.
- The host app must **not already own its own FCM message receiver** (e.g. another `FirebaseMessagingService`, or a Flutter/React Native FCM plugin). Android only delivers incoming pushes to one registered receiver per app — see *§8 Known limitations* before integrating into an app that already uses Firebase for push.

2. Installation

The library is published via JitPack.

Project-level (`settings.gradle` or top-level `build.gradle`, inside the `repositories { }` block):

```
maven { url 'https://jitpack.io' }
```

App-level (`app/build.gradle`):

```
dependencies {
    implementation 'com.github.Intellexal-Solutions-ITS:its-notification-library:<version>'
}
```

Replace `<version>` with the git tag you want to pull (check the

JitPack page

for which tags have successfully built). If you're consuming a version newer than what's already published, see *§9 Publishing a new version* below before relying on it.

3. Required manifest setup

The library does **not** self-register its FCM receiver in its own manifest — you must declare it explicitly in your app's `AndroidManifest.xml`:

```
<service
    android:name="com.its.notificationlibrary.Notifications.NotificationHandler"
    android:exported="false">
    <intent-filter>
        <action android:name="com.google.firebase.MESSAGING_EVENT" />
    </intent-filter>
</service>
```

The `POST_NOTIFICATIONS` permission is already declared by the library itself and will be merged into your app automatically — no action needed there, but you still need to **request** it at runtime on Android 13+ (see *§7*).

4. Initialize the SDK

Call this once, early — typically in your launcher `Activity`'s `onCreate` (or your `Application.onCreate`):

```
NotificationModule.initializeFirebase(
    context,
    firebaseApiKey,   // Firebase Web API key for ITS's project (ITS will give you this)
    clientId,         // Your client/tenant identifier, issued by ITS
    apiKey            // Your API key for ITS's backend, issued by ITS
);
```

What this does, in order:

- If your app has **no existing default** `FirebaseApp` (i.e. you have no `google-services.json` / no other Firebase usage), it initializes one using ITS's Firebase project and registers your device for FCM against that project.

- If your app **already has its own default** `FirebaseApp` (you already use Firebase for Analytics, Crashlytics, your own Auth, etc.), the library reuses it rather than creating a second one — Android's FCM client only supports one default `FirebaseApp` per process, so this is intentional, not a bug. See *§8* for what this means for push delivery in that scenario.
- Fetches the FCM device token and calls ITS's backend to issue your device an access token (stored internally, encrypted — see *§6*).

5. Register the user / device

After initialization (and once you have the user's identity, e.g. after a login or phone-verification step):

```
NotificationModule.registerUser(context, userPhone, userName, userEmail);
```

This refreshes the SDK's internal auth token first (transparently) and then registers the device against ITS's backend under that user's identity. Safe to call again later (e.g. if the user's name/email changes).

6. Token storage & security

The SDK stores its own bearer/refresh tokens and device fingerprint using `EncryptedSharedPreferences` internally — you don't need to (and shouldn't) manage these yourself. It also pins the TLS certificate for ITS's API host, so this stays protected even against a compromised device trust store. There's nothing for your app to configure here; it's mentioned so you know it's handled.

7. Runtime notification permission (Android 13+)

Request `POST_NOTIFICATIONS` yourself, from an `Activity`, before you expect notifications to display:

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU
        && ContextCompat.checkSelfPermission(this, Manifest.permission.POST_NOTIFICATIONS)
           != PackageManager.PERMISSION_GRANTED) {
    ActivityCompat.requestPermissions(this,
            new String[]{ Manifest.permission.POST_NOTIFICATIONS }, REQUEST_CODE);
}
```

8. Receiving notifications in-app (foreground listener)

If you want to react to a notification the moment it arrives (e.g. insert it into your own local database, show a custom in-app banner, update a badge count), register a listener:

```
NotificationHandlerRegistry.setListener(notification -> {
    String title = notification.getTitle();
    String message = notification.getMessage();
    JSONObject payload = notification.getPayload(); // raw FCM data payload
    // e.g. insert into your own local DB here
});
```

This fires every time `NotificationHandler` (the library's `FirebaseMessagingService`) receives a push, **in addition to** it showing the system notification and calling ITS's backend to mark it `"delivered"` — you don't need to do either of those yourself.

9. Handling notification taps (routing to a details screen)

This is the part that's easy to get wrong, because it involves two separate pieces wired together correctly. When the user taps the system notification, the library launches your app's launcher activity with a `String` extra named `"id"` (the backend's `transaction_id`).

Step 1 — register what should happen, before anything else touches the intent:

```
NotificationModule.setNotificationClickListener(id -> {
    // id == the transaction_id of the tapped notification.
    // Look it up (your own local store) and navigate to a details screen.
});
```

Step 2 — check the current intent for that `"id"` extra, in the same `Activity`,

after step 1:

```
NotificationModule.handleNotificationIntent(getIntent());
```

> **Order matters.** If you call `handleNotificationIntent` before `setNotificationClickListener`, the listener is still `null` at that point and the click is silently dropped. Always register the listener first.

Step 3 — if your launcher activity isn't the one with the details navigation logic

(e.g. you have a splash/login screen in front of your main screen), you must manually

forward the `"id"` extra when you redirect:

```
Intent intent = new Intent(this, NextActivity.class);
intent.putExtras(getIntent()); // <-- forwards the "id" extra (and anything else) along
startActivity(intent);
```

Skipping this step is the most common reason "notification tap doesn't open the details

screen" — the extra silently gets lost the moment any intermediate activity builds a

fresh `Intent` without copying it over.

10. Fetching notification history

```
NotificationModule.getNotifications(context, page, pageSize,
    new NotificationModule.NotificationCallBack() {
        @Override
        public void onSuccess(String responseBodyJson) {
            // Parse the JSON yourself — see the response shape below.
        }

        @Override
        public void onError(String errorMessage) { }
    });
```

The response is a JSON object: `{"status": "success", "data": [ {...}, ... ]}`, where

each item has (at least) `Id`, `Title`, `MsgData`, `Status`, `CreatedDatetime`. The

library does **not** persist this list for you — storing/caching it (e.g. in a local

Room database) and rendering it is entirely up to your app.

11. Marking a notification as read / updating its status

When the user opens a notification's details, tell ITS's backend about it:

```
NotificationUtil.updateNotificationStatus(context, "read", clientId, transactionId,
    new NotificationStatusCallback() {
        @Override
        public void onSuccess(String response) {
            // Also update your own local store's status to "read" here.
        }

        @Override
        public void onFailure(Exception e) { }
    });
```

Valid `status` values used elsewhere by the SDK itself are `"delivered"` (set automatically when the push arrives) and `"read"` (set by you, as above) — use the same two values so the backend and your own local store stay consistent.

12. Full integration checklist

- [ ] Added the JitPack repository + dependency.
- [ ] Added the `<service>` declaration for `NotificationHandler` in your manifest.
- [ ] Confirmed your app doesn't already run its own FCM receiver (see §8 below).
- [ ] Called `initializeFirebase(...)` once, early.
- [ ] Requested `POST_NOTIFICATIONS` at runtime (API 33+).
- [ ] Called `registerUser(...)` once you know who the user is.
- [ ] Registered a `NotificationHandlerRegistry` listener if you want in-app reactions.
- [ ] Registered a notification-click listener **before** calling `handleNotificationIntent`.
- [ ] Forwarded the `"id"` intent extra through every activity between the tap and your details screen, if there's more than one hop.
- [ ] Wired up `getNotifications(...)` + your own local storage for a history screen.
- [ ] Called `updateNotificationStatus(..., "read", ...)` when a notification is opened.

13. Known limitations

- **One `FirebaseMessagingService` per app, period.** Android delivers incoming FCM pushes to exactly one registered receiver in the whole app process. If your app already has its own (e.g. a Flutter app's `firebase_messaging` plugin, or a native app with its own push setup), adding this library's `NotificationHandler` as a second one

will not work reliably — one of them simply won't receive messages. There is currently no supported way around this; it needs to be resolved on a per-integration basis (either let the other receiver own FCM and skip this library's receiving path entirely, or vice versa).

- **FCM tokens are always scoped to your app's default Firebase project.** Firebase Cloud Messaging on Android has no secondary-app support — if your app already has its own Firebase project (for Analytics, Crashlytics, Auth, etc.), the token this library obtains belongs to **your** project, not ITS's. ITS's backend needs to be configured with credentials for your project for push delivery to actually work in that case; this is a backend-side setup question, not something fixed by a client code change.
- **The library doesn't persist notification history.** `getNotifications(...)` just wraps the HTTP call; storing results locally (for offline viewing, unread counts, etc.) is entirely your app's responsibility.

14. Publishing a new version (maintainers)

```
# bump the version in app/mylibrary/build.gradle's publishing { } block first
git add app/mylibrary/build.gradle
git commit -m "Bump library version to X.Y.Z"
git push origin main
git tag vX.Y.Z
git push origin vX.Y.Z
```

JitPack builds lazily on first request — after pushing the tag, trigger the build once by visiting `https://jitpack.io/#Intellexal-Solutions-ITS/its-notification-library/vX.Y.Z` or resolving it from a Gradle build, and check it went green before telling consumers to use that version.